

Desarrollo de firmware

Control de versiones: Git

Juan José Londoño, M.Sc.

Introducción

Git es un sistema de **control de versiones** distribuido que permite registrar, administrar y recuperar los cambios que se hacen en archivos de un proyecto a lo largo del tiempo.

En otras palabras, Git funciona como un historial inteligente del código, donde cada cambio que se realiza queda guardado y puede consultarse, compararse o revertirse.

¿Para qué se usa Git?

Git se usa principalmente para gestionar el desarrollo de software, permitiendo trabajar de manera ordenada y segura sobre el código.

Permite registrar cada modificación que se hace en los archivos del proyecto.

Si algo deja de funcionar, Git permite volver a una versión anterior que sí funcionaba.

¿Para qué se usa Git?

En proyectos profesionales varias personas trabajan sobre el mismo código. Git permite que cada desarrollador:

- haga cambios.
- Pruebe nuevas funciones.
- Integre su trabajo con el resto del equipo.
- Todo sin sobrescribir el trabajo de los demás.

Repositorios

Un repositorio es la “carpeta del proyecto” pero con historial, reglas y estructura para registrar cambios.

Un repo contiene:

- El código fuente (archivos .c y .h).
- El historial de versiones (**commits**).
- Metadatos de Git (en una carpeta oculta llamada .git).
- Otros documentos del proyecto (generalmente en formato .txt o .md).

Repositorio local y remoto

Repositorio local

Es el repositorio que está en el PC del desarrollador.

Aquí se escribe el código, es la “carpeta” que está guardada localmente con todos los archivos del proyecto.

Es el área de trabajo recurrente que usa el desarrollador.

Repositorio local y remoto

Repositorio remoto

Es una copia del repo que está en un servidor (internet o red de la empresa).

Ejemplos: GitHub, GitLab, Bitbucket, servidor interno.

Generalmente cuando se termina de trabajar en local se hace *push* al repo remoto.



Flujo de trabajo

El “ciclo de vida” de un cambio con Git tiene 3 áreas clave:

- A) *Working directory (carpeta local)*:** Donde se editan archivos y código (donde se trabaja con VS Code).
- B) *Staging area (área de preparación)*:** Es una zona intermedia donde se selecciona qué cambios entran al próximo *commit*.
- C) *Repository (historial de commits)*:** Cuando se hace un commit, esos cambios pasan a ser parte del historial.

¿Qué es un commit?

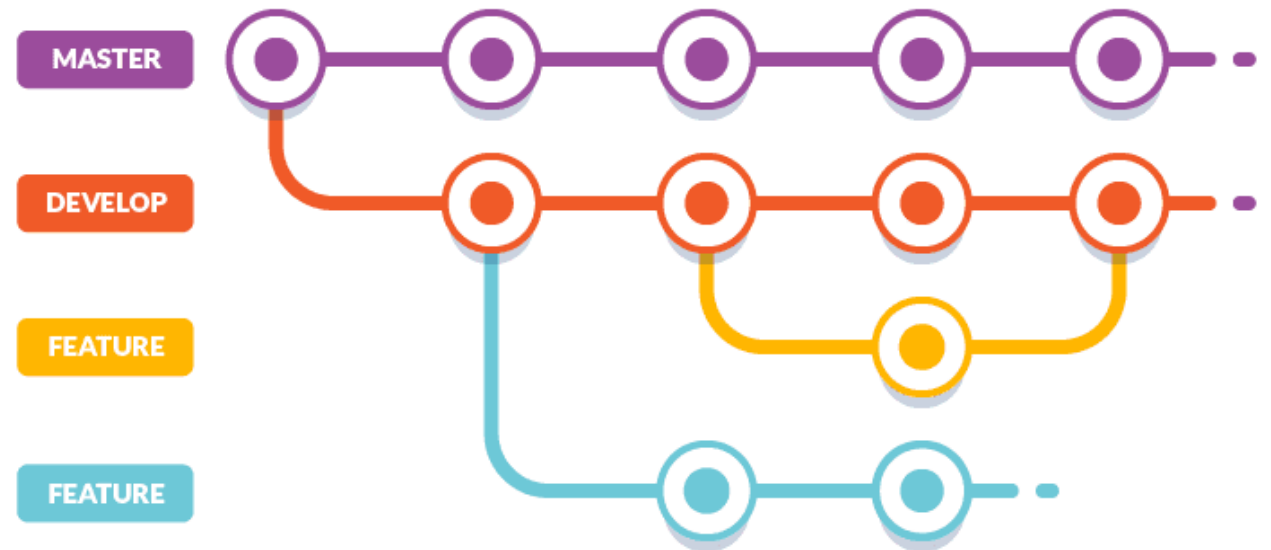
Es una “foto” del proyecto (más exactamente: un conjunto de cambios) guardada en el historial.

Incluye:

- Un identificador único (hash).
- Autor.
- Fecha.
- Mensaje.
- Cambios realizados

Ramas en Git (Branches)

Una rama es una línea de desarrollo independiente dentro de un repositorio. Permite trabajar en cambios o nuevas funcionalidades sin modificar directamente la versión principal del proyecto.



Pull requests

Un Pull Request es una solicitud para integrar los cambios de una rama en otra, normalmente hacia la rama principal.

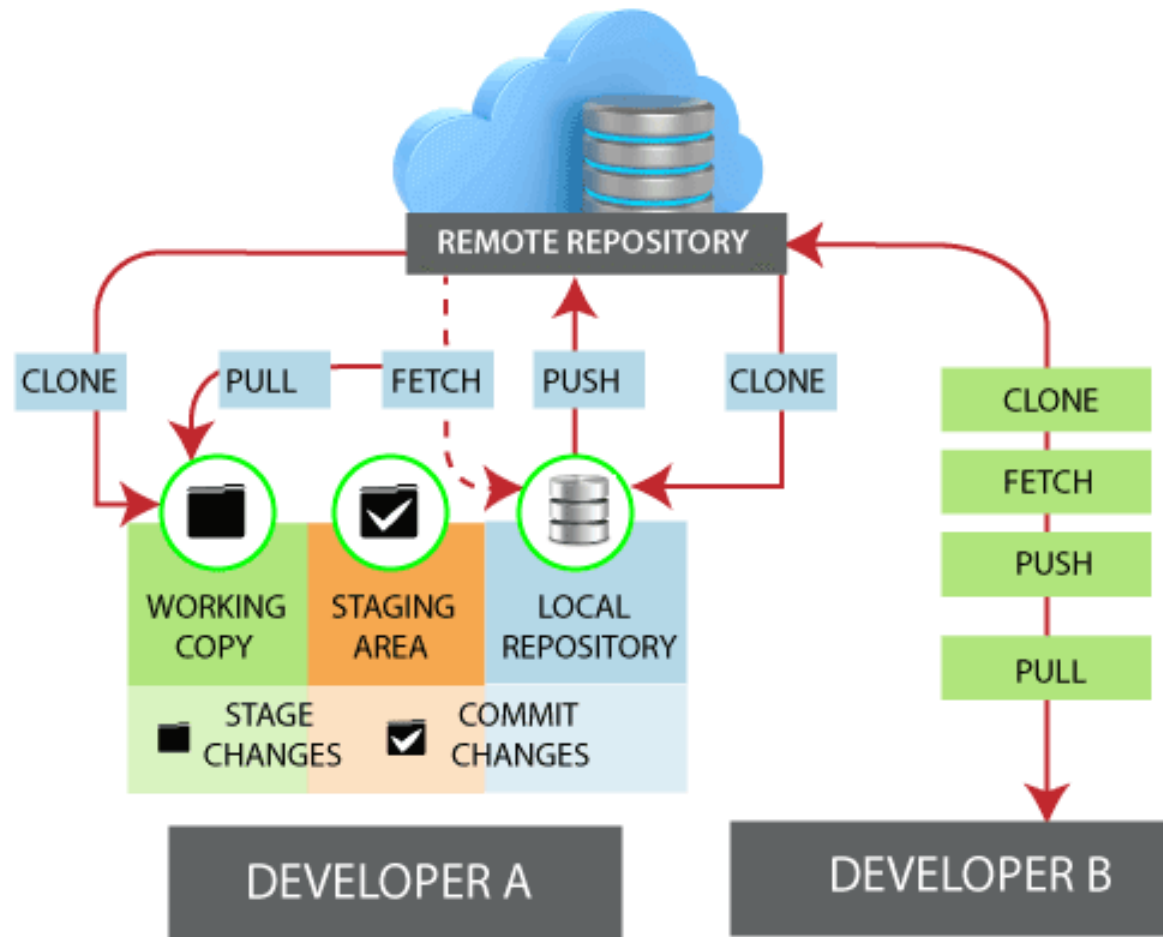
Un Pull Request pide que los cambios de una rama sean revisados y fusionados con la rama principal del proyecto.

Flujo de trabajo con pull requests

1. Clonar el repositorio remoto en la máquina local.
2. Cambiar a la rama principal del proyecto.
3. Crear una nueva rama para la funcionalidad o corrección que se va a desarrollar.
4. Realizar los cambios necesarios en el código.
5. Crear commits con los cambios realizados.

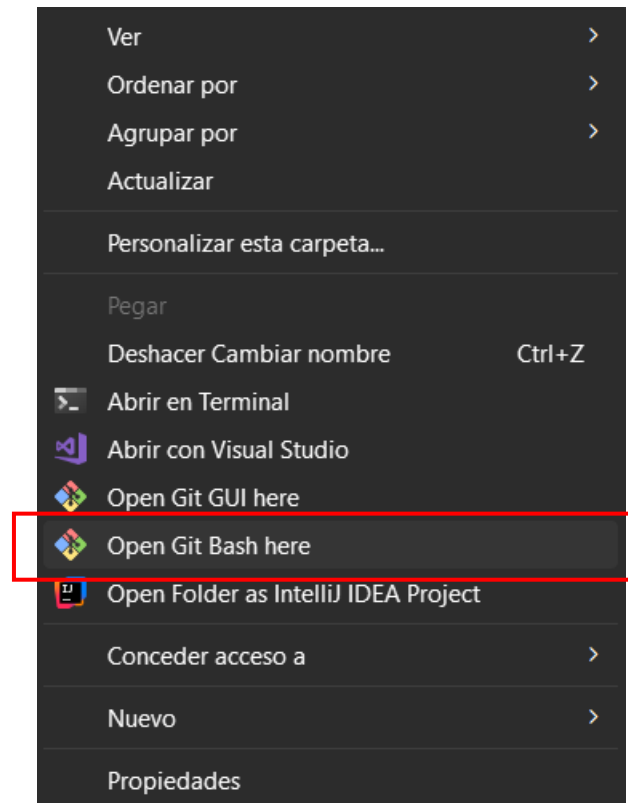
Flujo de trabajo con pull requests

6. Subir la nueva rama al repositorio remoto.
7. Crear un Pull Request en GitHub para solicitar la integración de la rama en la rama principal.
8. Esperar la revisión y aprobación del Pull Request.
9. Fusionar el Pull Request en la rama principal desde GitHub.
10. Actualizar la rama principal local con los cambios del repositorio remoto.
11. Eliminar la rama que ya fue integrada.



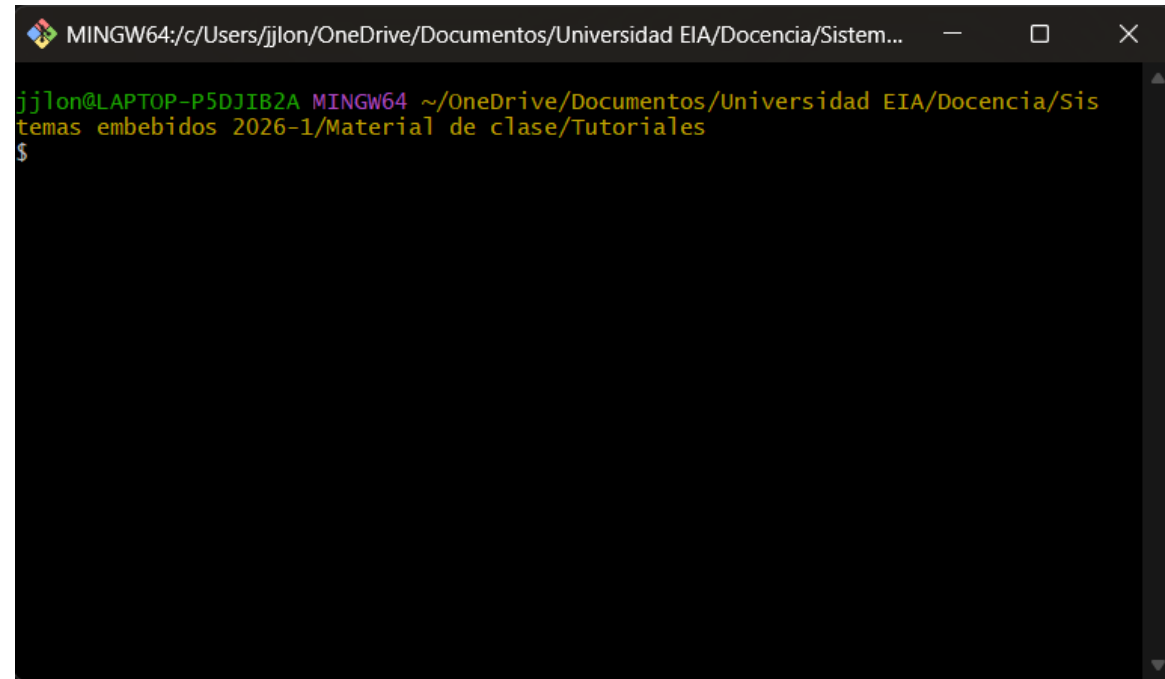
Uso de git desde la consola (Bash)

Dentro de la carpeta donde se desea iniciar un repositorio, hacer clic derecho y seleccionar “Open Git Bash here”.



Uso de git desde la consola (Bash)

Una vez en la consola de Git, todo el manejo se hará a través de comandos.



```
MINGW64:/c/Users/jjlon/OneDrive/Documentos/Universidad EIA/Docencia/Sistem...
jjlon@LAPTOP-P5DJIB2A MINGW64 ~/OneDrive/Documentos/Universidad EIA/Docencia/Sis
temas embebidos 2026-1/Material de clase/Tutoriales
$
```


Configuración inicial de Git

La primera vez que se usa Git es necesario configurar la identidad del usuario. Esta información se guarda en cada *commit* para saber quién realizó los cambios.



```
git config --global user.name "Nombre del usuario"
git config --global user.email "correo@ejemplo.com"
git config --list
```

Crear un nuevo repo local

Para comenzar un proyecto nuevo con Git se debe crear un repositorio. Una vez en la carpeta del proyecto, desde la terminal se ejecuta:

Este comando:

- inicializa un repositorio Git.
- Crea la carpeta oculta .git
- Apartir de ese momento la carpeta ya está siendo controlada por Git.

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text 'git init' is displayed in a light green monospace font.

Clonar un repo existente

Si el proyecto ya existe en un repositorio remoto, no se usa init. En ese caso se usa clone.

Este comando:

- Descarga el proyecto.
- Descarga todo el historial.
- Crea una copia local del repositorio.
- Conecta el repositorio local con el remoto.



```
git clone URL_DEL_REPOSITORIO
```

Ver el estado del repositorio

Este comando muestra:

- Archivos modificados.
- Archivos nuevos.
- Archivos listos para commit.
- Archivos que Git aún no está siguiendo.

Es el comando que más se usa cuando se trabaja con Git.



Agregar archivos al área de preparación

Antes de hacer un commit, los cambios deben agregarse al staging area.

Para agregar **un** archivo específico:

Para agregar **todos** los archivos:



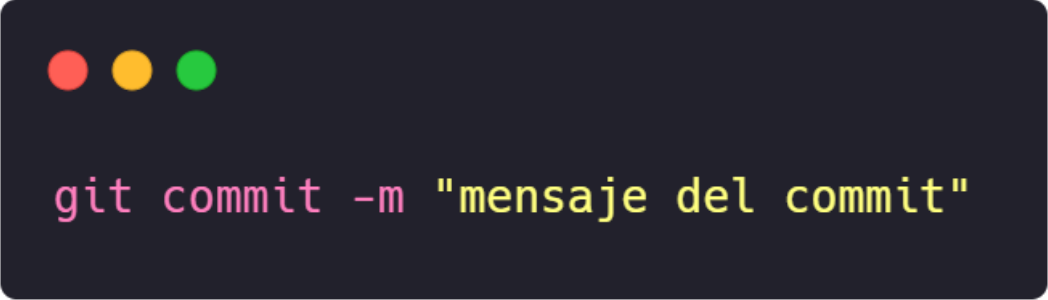
```
git add archivo.c
```



```
git add .
```

Crear un commit

Una vez agregados los cambios se crea un commit.

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the command `git commit -m "mensaje del commit"` in a light green monospace font.

```
git commit -m "mensaje del commit"
```

El mensaje debe describir claramente qué cambio se realizó. El commit guarda una nueva versión del proyecto en el historial.

Ver el historial de commits

Para ver los commits del repositorio:



```
git log
```

Manejo de ramas

Para crear una nueva rama. Cambiar a una rama. Crear y cambiar a la rama en un solo paso:



```
git branch nombre_rama  
git checkout nombre_rama  
git checkout -b nombre_rama
```


Registrar un repositorio remoto

Cuando se crea un repositorio con *git init*, el repositorio existe solo en la máquina local. Para poder sincronizarlo con un servidor remoto, es necesario registrar la dirección del repositorio remoto.



```
git remote add origin URL_DEL_REPOSITORIO
```

Subir cambios al repositorio remoto

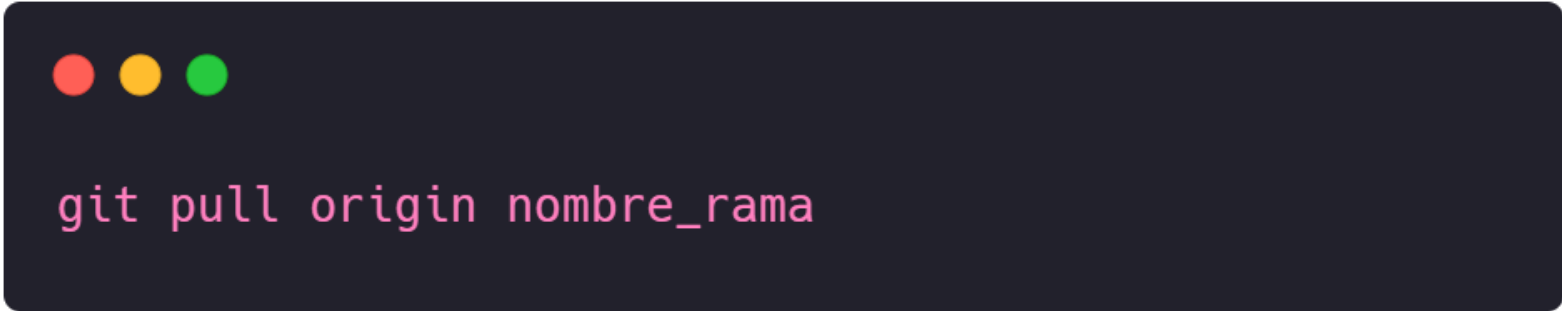
Después de hacer commits localmente, los cambios deben enviarse al repositorio remoto.



```
git push origin nombre_rama
```

Traer cambios del repositorio remoto

Para actualizar el repositorio local con los cambios del remoto se usa:



```
git pull origin nombre_rama
```

Eliminar una rama

Cuando una rama ya fue integrada se puede eliminar.



```
git branch -d nombre_rama
```

Resumen de comandos

```
-- Configuración inicial --  
  
git config --global user.name  
git config --global user.email  
  
-- Inicialización y clonación --  
git init  
git clone  
  
-- Trabajo diario --  
git status  
git add  
git commit  
git log  
  
-- Trabajo con ramas --  
git branch  
git checkout  
git switch  
  
-- Sincronización con remoto --  
git push  
git pull
```

Archivo .gitignore

es un archivo de configuración que le indica a Git qué archivos o carpetas no deben ser rastreados ni incluidos en el repositorio.

En muchos proyectos existen archivos que no deben versionarse, porque:

- Se generan automáticamente.
- Son temporales.
- Pueden ser muy pesados.
- Contienen información sensible.